

Reconocimiento activo:



TRABAJO REALIZADO POR:

Santiago Pérez Jiménez
2º DE INGENIERÍA DE LA CIBERSEGURIDAD

UNIVERSIDAD EUROPEA
MADRID
17 ABRIL 2026

PROFESOR: ALFREDO ROBLEDANO ABASOLO

1. Resumen:

Esta práctica trata de realizar un reconocimiento activo, para ello se trata de implementar estímulos adicionales a los que se vieron en clase, con el objetivo de descubrir hosts de maneras diferentes.

Eso es la primera parte, luego hay una segunda parte, que se tendrá que definir el funcionamiento por defecto de la herramienta “nmap” a la hora de descubrir puertos y estados.

Índice

1. Resumen:.....	2
2. Introducción:.....	5
3. Desarrollo:.....	6
Parte 1: Descubrimiento de hosts con scapy.....	6
Parte 2: Comportamiento por defecto nmap y port status.....	16
4. Resultados:.....	24
Parte 1 descubrimiento de hosts con scapy:.....	24
Parte 2: comportamiento por defecto de nmap:.....	25
5. Conclusiones:.....	27
6. Bibliografía:.....	28

Índice de figuras

Figura 1: Código descubrimiento de hosts 1.....	7
Figura 2: Código descubrimiento de host 2.....	8
Figura 3: Código descubrimiento de host 3.....	9
Figura 4: Docker compose 1.....	10
Figura 5: Docker compose 2.....	11
Figura 6: Docker compose 3.....	12
Figura 7: Ejecución docker compose 1.....	13
Figura 8: Ejecución docker compose 2.....	13
Figura 9: Interfaz.....	13
Figura 10: Output del código.....	14
Figura 11: Prueba Wireshark ARP 1.....	14
Figura 12: Prueba Wireshark ICMP, TCP y UDP 2.....	15
Figura 13: nmap localhost resultado.....	18
Figura 14: Conteo de paquetes enviados.....	19
Figura 15: Docker compose 4.....	20
Figura 16: Levantar docker compose.....	20
Figura 17: Resultado de análisis.....	21
Figura 18: Interfaz Wireshark.....	22
Figura 19: Análisis de puertos concreto.....	23
Figura 20: Resultado de Wireshark.....	23

2. Introducción:

El trabajo de hoy se realizará el reconocimiento activo en este caso de descubrimiento de hosts, puertos y sus estados, para ello se realizará un script y una investigación exhaustiva sobre el funcionamiento de nmap.

Este trabajo consta de dos partes:

1. Descubrimiento de hosts con scapy:

Nos piden que realicemos un script en el que creemos una función en python que nos permita realizar host discovery usando scapy con los protocolos UDP, TCP (ACK), ICMP (timestamp) y utilizarlo para detectar hosts activos.

2. Comportamiento por defecto nmap y port status:

Hay que investigar a fondo la herramienta, ejecutando por defecto nmap activa una serie de comportamientos preconfigurados que el auditor debe conocer a fondo, por lo tanto, hay que hacer unas breves definiciones de conceptos, identificar comportamientos por defectos de nmap y justificar dichos hallazgos con algún packet sniffer.

Entonces el objetivo final es implementar dichos estímulos adicionales que se vieron en clase de manera práctica para el descubrimiento de hosts. Además, definiendo el funcionamiento por defecto de la herramienta nmap en el descubrimiento de puertos y su estado.

3. Desarrollo:

Parte 1: Descubrimiento de hosts con scapy

El descubrimiento de hosts es fundamental en el proceso de auditoría activa de una red. El uso de estímulos y observar las respuestas permite a un atacante identificar dispositivos que exponen servicios para poder explotarlo después.

Por ello, me piden crear una función en Python que permite realizar el host discovery y tengo que utilizar scapy, me piden 3 protocolos UDP, TCP (ACK), ICMP (timestamp) y utilizarlo para detectar hosts activos.

Para ello tengo que realizar unas tareas:

- Crear una función “**craft_discovery_pkts**”, teniendo de argumentos:
 - Lista de 3 posibles protocolos en formato string o string que represente el protocolo a utilizar.
 - Rango de Ips en formato scapy o string que represente la IP a enviar el paquete.
 - Diccionario con claves de los protocolos y valores del número de paquetes a construir y si no se pasa solo se montará un paquete de cada tipo que sea exigido.
 - Puerto a utilizar en TCP y UDP, se usará el puerto 80 de no pasarse.
 - La función tiene que enviar al menos uno de los 3 protocolos, a un host activo y a otro que no tenga una IP con host que no lo tenga activa.
 - Se hará un scripto o jupyter notebook, que use la función mencionada junto a “sr” u otra función de scapy, que salga como “output” las Ips activas.

Código:

La primera parte del código se basa en el código con los argumentos obligatorios que se asignaron poniendo de opcional el del puerto 80 (esto es debido a que los protocolos TCP y UDP es necesario especificar un puerto de destino), luego si los protocolos son “string” lo modifiko a lista.

1. (Ivanov & Ivanov, n.d.)

```
from scapy.all import *
import ipaddress # usado para redes tipo 172.19.0.0/29

def craft_discovery_pkts(protocolos, ips, num_pkts=None, port=80):
    """
    Funcion que crea paquetes de descubrimiento de hosts activos usando ARP (si es local, si aplica) e ICMP, TCP y UDP

    Parametros:

    - protocolos: protocolos que se vaya a utilizar para el escaneo. Ej: ["ICMP"].
    - ips: IP o red de destino (ej: "10.0.1.10" o "10.0.1.0/24").
    - num_pkts: numero de paquetes por protocolo (si no se indica, se envia solo 1 por protocolo)
    - port: puerto destino para TCP y UDP (por defecto lo dejo en 80).

    Funcionamiento:

    ARP: funciona a nivel de redes locales, para resoluciones de direcciones MAC (nivel 2).
    ICMP: envia Timestamp Request para detectar los hosts activos.
    TCP: envia paquetes con flag ACK para provocar respuesta RST.
    UDP: envia datagramas y detecta respuesta ICMP si el puerto esta cerrado.

    Devuelve:

    Devuelve una lista de paquetes Scapy listos para ser enviados.

    """

    packets=[] # listado de paquetes que se van a enviar

    # si protocolos es string lo convierto a lista
    if isinstance(protocolos,str):
        protocolos=[protocolos]
```

Figura 1: Código descubrimiento de hosts 1

El código se basa en dos partes, la primera es la construcción de los paquetes de ICMP, UDP y TCP.

Luego la condición de especificar el número de paquetes y si no se especifica solo se envía uno.

ICMP, UDP y TCP, cumplen con las condiciones haciendo que TCP actúe mediante ACK, mediante la flag “A”.

En la construcción del paquete se hace lo siguiente:

- Se ve si existe dicho protocolo, se repite el proceso dependiendo del número de paquetes a enviar, dentro construyo la capa 3, que es la IP de destino y la capa 4 la uso para cada protocolo con lo exigido, luego se construye el paquete con la capa 3 y 4, y guardo el paquete.

```
# si no se especifica el numero de paquetes
if num_pkts is None:

    num_pkts={}

    for proto in protocolos:

        num_pkts[proto]=1 # envio uno por protocolo

# ICMP TIMESTAMP
if "ICMP" in protocolos: # si esta activo ICMP

    for i in range(num_pkts["ICMP"]): # repite el envio N veces dependiendo de "num_pkts"

        l3 = IP(dst=ips) # capa 3, IP de destino (siendo posible IP o red)

        l4 = ICMP(type=l3) # capa 4, ICMP Timestamp Request, para ver si el host responde ICMP

        pkt = l3 / l4 # construyo paquete

        packets.append(pkt) # guardo paquete

# UDP DISCOVERY
if "UDP" in protocolos:

    for i in range(num_pkts["UDP"]):

        l3 = IP(dst=ips)

        l4 = UDP(dport=port) # envio UDP al puerto 80 o el que se pase.

        pkt = l3 / l4

        packets.append(pkt)

# TCP ACK DISCOVERY
if "TCP" in protocolos:

    for i in range(num_pkts["TCP"]):

        l3 = IP(dst=ips)

        l4 = TCP(dport=port,flags="A") # es ACK debido al flag que la flag es la A

        pkt = l3 / l4

        packets.append(pkt)
```

Figura 2: Código descubrimiento de host 2

Esta es la segunda parte, en ella se define los protocolos, los “targets” a los que se escanea las IPs, y se dice cuál es la red local.

La red local, se supone que la se yo, por eso establezco ese rango.

Para ambas varias se puede poner o una IP, una serie de IPs en lista o un rango de IP.

La separación de entre ARP y los otros 3 protocolos es muy sencillo de explicar, normalmente no se podrían utilizar porque trabajan en otras capas, si se imagina enviando a una red local los otros 3 protocolos se haría sin problema, pero antes el sistema normalmente hará ARP internamente para conocer la MAC de destino.

Es más sencillo de lo que parece, ARP trabaja en la capa 2, lo que hace es obtener la MAC de una IP en la LAN, ICMP trabaja con la capa 3, lo que hace son mensajes de control o hace diagnósticos y luego está TCP y UDP que trabajan en la capa 4 que lo que hacen es la comunicación entre aplicaciones.

2.(Peiro, 2026)

Entonces, ¿Qué pasa si por un casual en “red_local” pongo una IP o rango de IP que no está en la red local? Muy sencillo la propia consola nos haría una alerta de que localiza la MAC y entonces puede funcionar, pero puede incluso llegar a fallar.

Luego el código se construye de un bucle para analizar todas las IP de una en una, lo que hacemos es una condición para determinar si pertenece a la “red_local” o no.

Si pertenece a ARP, lo que hace es construye el paquete compuesto por el destino del broadcast Ethernet, básicamente lo que hace es enviar a todos los equipos de la red local, pregunta por una IP concreta (ARP(pdst=ip)) y luego scapy envía ese frame.

Si pertenece a alguno de los otros 3 protocolos, lo que hace es generar los paquetes junto a su paquete IP y luego lo envía a la IP de destino.

```
return packets # devuelvo todos los paquetes listos

protocolos=["ICMP","TCP","UDP"]
target_ips=["172.19.0.2","172.19.0.3","10.0.1.10","10.0.2.10"] # lista de objetivos a examinar
red_local = "172.19.0/29" # defino cuales son locales, decido si se usa ARP
ans = [] # lista para guardar las respuestas
for ip in target_ips: # bucle para analizar una ip de una
    # si es local uso ARP
    if ipaddress.ip_address(ip) in ipaddress.ip_network(red_local): # miro si la ip esta dentro de la red local
        arp = Ether(dst="ff:ff:ff:ff:ff:ff") / ARP(pdst=ip) # broadcast ARP
        r = srp(arp, timeout=2, verbose=0, iface="br-3f020f3d3088")[0] # capa 2, envio a esa capa y se descubre los host locales reales
        ans += r # guardo las respuestas
    # si no es local
    else:
        pkts = craft_discovery_pkts(protocolos, ip) # genero paquetes ICMP, TCP y UDP
        for pkt in pkts:
            r = sr(pkt, timeout=1, verbose=0)[0] # envio a capa 3
            ans += r

hosts_activos = set() # evito los duplicados con set
for snd, rcv in ans: # snd es lo que envíe, y rcv es la respuesta
    if rcv.haslayer(ARP): # si es ARP
        hosts_activos.add(rcv.psrc) # responde con la ip
    else:
        hosts_activos.add(snd[IP].dst) # si no lo es, responde con la IP de destino original

print("Hosts activos detectados:")
for ip in hosts_activos: # muestro los hosts activos sin duplicado
    print(ip)
```

Figura 3: Código descubrimiento de host 3

Utilizaré dos docker compose de prueba, uno de una red local y otro en el que se distingue dos redes distintas, para poder probar la teoría del ARP y de los otros 3 protocolos.

Es importante destacar que aunque use el otro docker compose con dos redes distintas está sigue siendo “red local” por estar con docker, pero aun así la prueba debería de salir correctamente, pero puede que se cuele el protocolo de ARP.

Docker compose que usaré de prueba para el protocolo ARP:

```
docker-compose2.yml
1  services:
2
3    ssh:
4      image: rastasheep/ubuntu-sshd
5      container_name: lab_ssh
6      environment:
7        - USER_NAME=ssh-user
8        - LISTEN_PORT=22
9        - PASSWORD_ACCESS=true
10       - USER_PASSWORD=password123
11      ports:
12        - "2222:22"
13
14      networks:
15        - custom-network
16
17    ftp:
18      image: fauria/vsftpd
19      container_name: lab_ftp
20      environment:
21        - FTP_USER=kali
22        - FTP_PASS=test123
23      ports:
24        - "21:21"
25      networks:
26        - custom-network
27
28    web:
29      image: nginx:latest
30      container_name: lab_http
31      ports:
32        - "8080:80"
33      networks:
34        - custom-network
35
36    networks:
37      custom-network:
38        driver: bridge
39
40
```

Figura 4: Docker compose 1

Se compone de 3 servicios, SSH, FTP y HTTP.

El otro docker compose que utilizaré para probar los protocolos TCP, UDP, ICMP:

```
! multi-hop.yml x  docker-compose2.yml  Untitled-1  docker-composeDNS.yml
! multi-hop.yml
1  services:
2      # --- ENDPOINTS ---
3      app1:
4          image: alpine:latest
5          container_name: app1
6          cap_add: [NET_ADMIN]
7          networks:
8              net1:
9                  ipv4_address: 10.0.1.10
10             command: sh -c "ip route replace default via 10.0.1.254 && sleep infinity"
11
12      app2:
13          image: alpine:latest
14          container_name: app2
15          cap_add: [NET_ADMIN]
16          networks:
17              net2:
18                  ipv4_address: 10.0.2.10
19             command: sh -c "ip route replace default via 10.0.2.254 && sleep infinity"
20
21      # --- ROUTER 1 (Edge Router A) ---
22      router-1:
23          image: alpine:latest
24          container_name: router-1
25          cap_add: [NET_ADMIN]
26          sysctls:
27              - net.ipv4.ip_forward=1
28          networks:
29              net1:
30                  ipv4_address: 10.0.1.254
31              net12:
32                  ipv4_address: 10.0.12.1
33             # Needs to know that Net-2 is accessible via Router-2
34             command: sh -c "ip route add 10.0.2.0/24 via 10.0.12.2 && sleep infinity"
35
36      # --- ROUTER 2 (The Middleman) ---
37      router-2:
38          image: alpine:latest
39          container_name: router-2
40          cap_add: [NET_ADMIN]
41          sysctls:
42              - net.ipv4.ip_forward=1
43          networks:
44              net12:
45                  ipv4_address: 10.0.12.2
46              net23:
47                  ipv4_address: 10.0.23.1
48             command: >
49                 sh -c "ip route add 10.0.1.0/24 via 10.0.12.1 &&
50                     ip route add 10.0.2.0/24 via 10.0.23.2 &&
51                     sleep infinity"
```

Figura 5: Docker compose 2

En este caso se divide en 2 máquinas con “Alpine” y de 3 routers.

```

# --- ROUTER 3 (Edge Router B) ---
router-3:
  image: alpine:latest
  container_name: router-3
  cap_add: [NET_ADMIN]
  sysctls:
    - net.ipv4.ip_forward=1
  networks:
    net23:
      ipv4_address: 10.0.23.2
    net2:
      ipv4_address: 10.0.2.254
  # Needs to know that Net-1 is accessible via Router-2
  command: sh -c "ip route add 10.0.1.0/24 via 10.0.23.1 && sleep infinity"

networks:
  net1:
    ipam:
      config: [{subnet: 10.0.1.0/24, gateway: 10.0.1.1}]
  net12:
    ipam:
      config: [{subnet: 10.0.12.0/24, gateway: 10.0.12.254}] # Host takes .254 here
  net23:
    ipam:
      config: [{subnet: 10.0.23.0/24, gateway: 10.0.23.254}] # Host takes .254 here
  net2:
    ipam:
      config: [{subnet: 10.0.2.0/24, gateway: 10.0.2.1}]

```

Figura 6: Docker compose 3

Prueba de ejecución:

Lo primero que hago es levantar ambos docker compose:

```
$ docker compose -f docker-compose2.yml up -d
[+] Running 3/3
 ✓ Container lab_http Started
 ✓ Container lab_ssh Started
 ✓ Container lab_ftp Started
```

Figura 7: Ejecución docker compose 1

```
[+] Running 5/5
 ✓ Container app1 Started
 ✓ Container app2 Started
 ✓ Container router-1 Started
 ✓ Container router-2 Started
 ✓ Container router-3 Started
```

Figura 8: Ejecución docker compose 2

Luego verifico cuál es la interfaz para poder ponerlo en el script y poder detectar con el ARP, el del otro docker compose, lo miro directamente en Wireshark que utilizaré para que efectivamente me hace lo que estoy buscando.

```
5: br-3f020f3d3088: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 06:a0:4d:25:62:6c brd ff:ff:ff:ff:ff:ff
    inet 172.19.0.1/16 brd 172.19.255.255 scope global br-3f020f3d3088
        valid_lft forever preferred_lft forever
    inet6 fe80::4a0:4dff:fe25:626c/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Figura 9: Interfaz

Primero haré la prueba con el ARP, como se ve, funciona correctamente me saca las IPs de ambos protocolos, a primera vista parece que no sabemos si funciona y por ello utilizamos Wireshark para verificar que hace lo que queremos:

```

protocolos=["ICMP","TCP","UDP"]

target_ips=["172.19.0.2","172.19.0.3","10.0.1.10","10.0.2.10"]

red_local = "172.19.0.0/29" # defino cuales son locales, decido

ans = [] # lista para guardar las respuestas

for ip in target_ips: # bucle para analizar una ip de una

    # si es local uso ARP
    if ipaddress.ip_address(ip) in ipaddress.ip_network(red_local):

        arp = Ether(dst="ff:ff:ff:ff:ff:ff") / ARP(pdst=ip) # b

        r = srp(arp, timeout=2, verbose=0, iface="br-3f020f3d39")
        ans += r # guardo las respuestas

    # si no es local
    else:

        pkts = craft_discovery_pkts(protocolos, ip) # genero pa

        for pkt in pkts:
            r = sr(pkt, timeout=1, verbose=0)[0] # envio a capa
            ans += r

hosts_activos = set() # evito los duplicados con set

for snd, rcv in ans: # snd es lo que envíe, y rcv es la respue

    if rcv.haslayer(ARP): # si es ARP
        hosts_activos.add(rcv.psrc) # responde con la Ip
    else:
        hosts_activos.add(snd[IP].dst) # si no lo es, responde

print("Hosts activos detectados:")

for ip in hosts_activos: # muestro los hosts activos sin duplica
    print(ip)

```

✓ 9.2s

Hosts activos detectados:
10.0.1.10
172.19.0.3
172.19.0.2
10.0.2.10

Figura 10: Output del código

Este sería el Wireshark para verificar que me hace solo el ARP de la red local.

Lo que dice ahí es lo siguiente:

El equipo 172.19.0.1 está preguntando en broadcast quien tiene la IP 172.19.0.2 y que le responda, entonces dicha IP responde y le da su MAC, entonces la IP 172.19.0.1, ya conoce la MAC del otro.

Luego se ejecuta la misma operación, pero el que acaba en “.3”:

Apply a display filter ... <Ctrl-/>						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	06:a0:4d:25:62:6c	Broadcast	ARP	42	Who has 172.19.0.2? Tell 172.19.0.1
2	0.000047387	86:9b:03:c3:ee:5c	06:a0:4d:25:62:6c	ARP	42	172.19.0.2 is at 86:9b:03:c3:ee:5c
3	0.039919403	06:a0:4d:25:62:6c	Broadcast	ARP	42	Who has 172.19.0.3? Tell 172.19.0.1
4	0.039971145	2e:31:ae:b4:c5:47	06:a0:4d:25:62:6c	ARP	42	172.19.0.3 is at 2e:31:ae:b4:c5:47

Figura 11: Prueba Wireshark ARP 1

Ahora haríamos la misma operación en Wireshark abro el de la interfaz con dicha IP para verificarlo.

Tenemos el host origen que es el 10.0.1.1 y comprueba si el host 10.0.1.10 está activo, para comprobar esto hace 3 técnicas distintas.

Envía un primer paquete, este paquete es una solicitud ICMP tipo 13, le dice que si está activo que responda con la hora, no es un ping clásico es un “Timestamp Request”, como pedía el ejercicio.

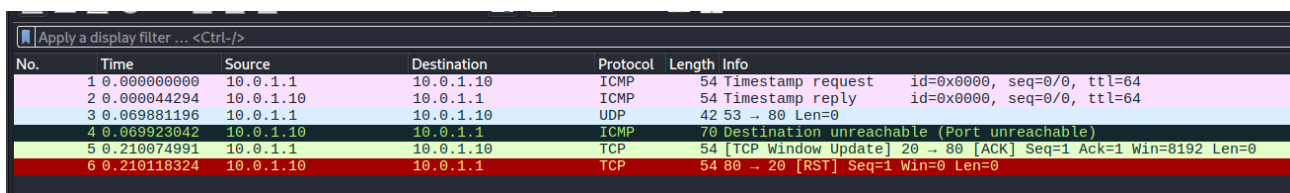
El segundo paquete el host de destino responde.

El tercer paquete envía un datagrama UDP vacío al puerto 80 (el problema es que aquí Scapy usa puertos aleatorios y sale como si fuera el 53).

El cuarto paquete responde que está vivo, pero que el puerto UDP está cerrado.

El quinto paquete envía un paquete TCP con la flag de “ACK”, sin conexión previa, hace un “ACK scan”.

El sexto paquete es el host respondiendo que no hay una conexión abierta y entonces resetea, haciendo ver que el TCP funciona.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.1.1	10.0.1.10	ICMP	54	Timestamp request id=0x0000, seq=0/0, ttl=64
2	0.000044294	10.0.1.10	10.0.1.1	ICMP	54	Timestamp reply id=0x0000, seq=0/0, ttl=64
3	0.069881196	10.0.1.1	10.0.1.10	UDP	42	53 → 80 Len=0
4	0.069923042	10.0.1.10	10.0.1.1	ICMP	70	Destination unreachable (Port unreachable)
5	0.210074991	10.0.1.1	10.0.1.10	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
6	0.210118324	10.0.1.10	10.0.1.1	TCP	54	80 → 20 [RST] Seq=1 Win=0 Len=0

Figura 12: Prueba Wireshark ICMP, TCP y UDP 2

Parte 2: Comportamiento por defecto nmap y port status

Nmap (Network Mapper) es una herramienta muy popular, en la industria para la administración de redes y en la auditoría de seguridad. Simplemente ejecutando nmap con su configuración predefinida, se activa una serie de comportamientos preconfigurados que el auditor debe conocer a fondo.

Para ello voy a realizar una serie de tareas que nos han definido:

1. Definir brevemente el concepto de estado de puerto y el tipo de estímulos y respuestas (campos de paquete) que permiten determinar su estado (abierto, cerrado, filtrado).

El concepto de estado de puerto nos indica si un servicio en una máquina remota está accesible o no desde la red. Nmap determina este estado enviando paquetes de red específicos (TCP y/o UDP) y analiza las respuestas del sistema objetivo.

Sus estados principales, los del puerto son los siguientes:

- Abierto:
 - El puerto está escuchando conexiones activamente y existe un servicio que responde a peticiones o respuestas que le lleguen.
 - Su respuesta típica en TCP sería **“SYN-ACK”**.
- Cerrado:
 - El puerto es accesible, y es capaz de recibir respuestas, pero no hay ningún servicio escuchando, el cerrado si responde a los sondeos de nmap, lo que permite confirmar que dicho host está activo y accesible.
 - Su respuesta típica en TCP sería **“RST”**.
- Filtrado:
 - Un firewall o un filtro de red bloquea los paquetes, por lo que nmap no puede determinar si está abierto o cerrado, este no recibe respuestas y no puede determinarse si es accesible porque un firewall bloquea el tráfico.
 - Su respuesta es que no hay respuesta o suele recibir **“ICMP unreachable”**.

El tipo de estímulos y respuestas que usa nmap sería las siguientes:

Nmap envía principalmente lo siguiente:

- SYN (TCP SYN scan) → está intentando iniciar conexión.
- ACK / FIN / NULL / XMAS → son técnicas de escaneo alternativas usadas para inferir en los estados de puertos y, en algunos casos, sortear ciertos filtros.
- UDP packets → esto es para servicios UDP

Las respuestas son las vistas anteriormente para puerto abierto “SYN-ACK”, para puerto cerrado “RST” y para el puerto filtrado o no tiene respuesta o “ICMP blocked”

2. Identificar el comportamiento por defecto de nmap para el reconocimiento de puertos abiertos.

Para identificar el comportamiento por defecto de nmap voy a realizar el comando de “nmap localhost”:

1. Conteo de puertos:

Para identificar el comportamiento por defecto de nmap voy a realizar el comando de “nmap localhost” y me sirve también para el conteo de puertos:

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-23 04:09 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000050s latency).
Other addresses for localhost (not scanned): ::1
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds
```

Figura 13: nmap localhost resultado

Este escaneo me dice varias cosas, el host está activo o “up”, la máquina responde, luego escaneó 1000 puertos TCP más comunes, pero ningún servicio está escuchando en esos puertos en el momento del escaneo y el método de detección el “reset”, nos indica que los puertos cerrados respondieron con paquetes TCP RST.

2. Conteo de paquetes enviados.

```
l-$ nmap --packet-trace localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-23 04:17 -0400
SENT (0.0355s) TCP 127.0.0.1:53647 > 127.0.0.1:554 S ttl=51 id=52127 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:587 S ttl=44 id=20927 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:3306 S ttl=46 id=12758 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:8080 S ttl=41 id=7814 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:110 S ttl=57 id=17333 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:111 S ttl=47 id=60857 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:1025 S ttl=39 id=24686 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0358s) TCP 127.0.0.1:53647 > 127.0.0.1:199 S ttl=45 id=45134 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0358s) TCP 127.0.0.1:53647 > 127.0.0.1:135 S ttl=41 id=59136 iplen=44 seq=3398025168 win=1024 <mss 1460>
SENT (0.0358s) TCP 127.0.0.1:53647 > 127.0.0.1:139 S ttl=54 id=64435 iplen=44 seq=3398025168 win=1024 <mss 1460>
RCVD (0.0355s) TCP 127.0.0.1:53647 > 127.0.0.1:554 S ttl=51 id=52127 iplen=44 seq=3398025168 win=1024 <mss 1460>
RCVD (0.0355s) TCP 127.0.0.1:554 > 127.0.0.1:53647 RA ttl=64 id=0 iplen=40 seq=0 win=0
RCVD (0.0356s) TCP 127.0.0.1:53647 > 127.0.0.1:587 S ttl=44 id=20927 iplen=44 seq=3398025168 win=1024 <mss 1460>
RCVD (0.0357s) TCP 127.0.0.1:587 > 127.0.0.1:53647 RA ttl=64 id=0 iplen=40 seq=0 win=0
RCVD (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:3306 S ttl=46 id=12758 iplen=44 seq=3398025168 win=1024 <mss 1460>
RCVD (0.0357s) TCP 127.0.0.1:3306 > 127.0.0.1:53647 RA ttl=64 id=0 iplen=40 seq=0 win=0
RCVD (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:8080 S ttl=41 id=7814 iplen=44 seq=3398025168 win=1024 <mss 1460>
RCVD (0.0357s) TCP 127.0.0.1:8080 > 127.0.0.1:53647 RA ttl=64 id=0 iplen=40 seq=0 win=0
RCVD (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:110 S ttl=57 id=17333 iplen=44 seq=3398025168 win=1024 <mss 1460>
RCVD (0.0357s) TCP 127.0.0.1:110 > 127.0.0.1:53647 RA ttl=64 id=0 iplen=40 seq=0 win=0
RCVD (0.0357s) TCP 127.0.0.1:53647 > 127.0.0.1:111 S ttl=47 id=60857 iplen=44 seq=3398025168 win=1024 <mss 1460>
```

Figura 14: Conteo de paquetes enviados

El parámetro que uso muestra cada paquete enviado y recibido por nmap durante el escaneo.

Ejemplo de paquete enviado:

“SENT TCP 127.0.0.1:53647 > 127.0.0.1:554 S”

- Esto significa que nmap usa un puerto origen aleatorio (53647)
- Envía un paquete TCP SYN.
- El destino del puerto es 554.
- La conclusión es que intenta comprobar si ese puerto está abierto.

Ejemplo de respuesta recibida:

“RCVD TCP 127.0.0.1:554 > 127.0.0.1:53647 RA”

- Esto significa que el puerto responde con “RST + ACK”, indicando que el puerto está cerrado.

Nmap hace lo siguiente:

Envía SYN y espera respuesta. Si recibe “SYN-ACK”, está abierto, si recibe “RST-ACK”, está cerrado y si no recibe nada está filtrado.

En este caso responde con RA, por eso salen que están cerrados.

Sale el escaneo de 1000 puertos y me salía 1000 salidas por lo que interpreto que se envía paquete por puerto escaneado, en resumen son 1000 paquetes SYN enviados por puerto.

3. Dar ejemplos con al menos dos servicios expuestos (y el caso de puerto cerrado), para esta parte la realizaré con un “docker compose” con tres servicios activos.
3.(Inc, 2026)

Dicho docker compose es el siguiente:

```
docker-compose2.yml
1  services:
2
3      ssh:
4          image: rastasheep/ubuntu-sshd
5          container_name: lab_ssh
6          environment:
7              - USER_NAME=ssh-user
8              - LISTEN_PORT=22
9              - PASSWORD_ACCESS=true
10             - USER_PASSWORD=password123
11          ports:
12              - "2222:22"
13
14          networks:
15              - custom-network
16
17      ftp:
18          image: fauria/vsftpd
19          container_name: lab_ftp
20          environment:
21              - FTP_USER=kali
22              - FTP_PASS=test123
23          ports:
24              - "21:21"
25          networks:
26              - custom-network
27
28      web:
29          image: nginx:latest
30          container_name: lab_http
31          ports:
32              - "8080:80"
33          networks:
34              - custom-network
35
36  networks:
37      custom-network:
38          driver: bridge
39
```

Figura 15: Docker compose 4

Tengo 3 servicios, uno de SSH, otro de FTP y el último de HTTP

Para ello levanto este docker compose primero y verifico que se haya levantado de forma correcta:

```
$ docker compose -f docker-compose2.yml up -d
[+] Running 3/3
 ✓ Container lab_ssh   Started
 ✓ Container lab_ftp   Started
 ✓ Container lab_http  Started

(kali@kali)~[~/Desktop/pruebas-compose]
$ docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                               NAMES
98930807c631   fauria/vsftpd        "/usr/sbin/run-vsftpd..." 7 seconds ago  Up 6 seconds  20/tcp, 0.0.0.0:21→21/tcp, [::]:21→21/tcp  lab_ftp
e28a46c97f71   rastasheep/ubuntu-sshd "/usr/sbin/sshd -D"       7 seconds ago  Up 6 seconds  0.0.0.0:2222→22/tcp, [::]:2222→22/tcp  lab_ssh
2b619414541f   nginx:latest         "/docker-entrypoint..." 7 seconds ago  Up 6 seconds  0.0.0.0:8080→80/tcp, [::]:8080→80/tcp  lab_http
```

Figura 16: Levantar docker compose

Tras comprobar que funciona de forma correcta iré a hacer el análisis de dichos puertos en concreto más el puerto cerrado:

```
L$ nmap -p 21,23,2222,8080 localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-23 05:14 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000097s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE
21/tcp    open  ftp
23/tcp    closed telnet
2222/tcp  open  EtherNetIP-1
8080/tcp  open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
```

Figura 17: Resultado de análisis

Como se aprecia los 3 servicios que he levantado se verían como “open”, y están listos para escuchar y el otro el 23 no está levantado, luego como dato curioso el puerto 2222, en realidad se asigna a otro, pero en docker compose lo he mapeado en dicho puerto, eso nmap no sabe diferenciarlo, el solo lo asigna con la base datos que tiene.

3. Justificar los hallazgos mediante algún packet sniffer (scapy, wireshark, tcpdump...), como dato nos dicen que solo nos fijaremos en el reconocimiento de puertos, descartando el tráfico de nmap para el descubrimiento y resolución de nombre de hosts activos.

Abro el Wireshark y selecciono la interfaz correspondiente al docker compose:

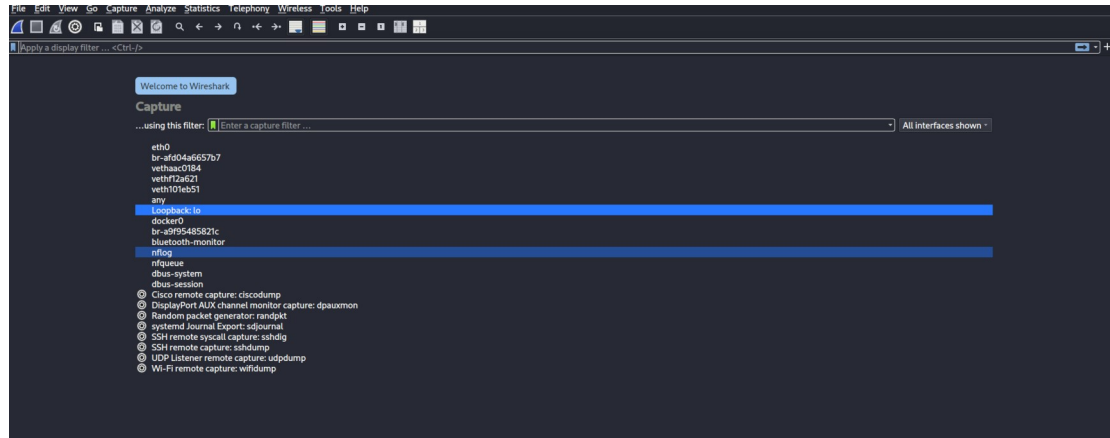


Figura 18: Interfaz Wireshark

Realizamos el comando correspondiente con nmap filtrando el descubrimiento de host, para ello usaré el parámetro “-Pn” que me ayuda a evitar el descubrimiento de host:

```
L$ nmap -Pn -p 21,23,2222,8080 localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-23 05:38 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00016s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE
21/tcp    open  ftp
23/tcp    closed telnet
2222/tcp  open  EtherNetIP-1
8080/tcp  open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

Figura 19: Análisis de puertos concreto

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	58	56835 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2	0.000079251	127.0.0.1	127.0.0.1	TCP	58	21 → 56835 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
3	0.000091650	127.0.0.1	127.0.0.1	TCP	54	56835 → 21 [RST] Seq=1 Win=0 Len=0
4	0.000128286	127.0.0.1	127.0.0.1	TCP	58	56835 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	0.000138424	127.0.0.1	127.0.0.1	TCP	58	8080 → 56835 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
6	0.000141633	127.0.0.1	127.0.0.1	TCP	54	56835 → 8080 [RST] Seq=1 Win=0 Len=0
7	0.000150803	127.0.0.1	127.0.0.1	TCP	58	56835 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
8	0.000154639	127.0.0.1	127.0.0.1	TCP	54	23 → 56835 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9	0.000163145	127.0.0.1	127.0.0.1	TCP	58	56835 → 2222 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	0.000169159	127.0.0.1	127.0.0.1	TCP	58	2222 → 56835 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
11	0.000171432	127.0.0.1	127.0.0.1	TCP	54	56835 → 2222 [RST] Seq=1 Win=0 Len=0

Figura 20: Resultado de Wireshark

He utilizado Wireshark como herramienta de análisis de tráfico durante la ejecución de nmap sobre el host local.

Durante el escaneo de puertos se observa paquetes TCP de tipo “SYN” enviados por nmap hacia los puertos que he seleccionado en concreto. En los puertos 21, 2222 y 8080, se recibieron respuestas “SYN-ACK”, lo que indica que dichos puertos se encuentran abiertos y existen servicios escuchando.

Posteriormente, nmap finaliza la conexión mediante el envío de paquetes RST. En el caso del puerto 23 se ve que la respuesta “RST-ACK” al instante del SYN, indicando que está cerrado el puerto.

4. Resultados:

Parte 1 descubrimiento de hosts con scapy:

1. Resultado con ARP:

Tipo de prueba	IP objetivo	Protocolo usado	Respuesta recibida	Resultado	Interpretación
Red local Docker	172.19.0.2	ARP	172.19.0.2 is at ...	Activo	Host responde ARP y existe en la LAN
Red local Docker	172.19.0.3	ARP	172.19.0.3 is at ...	Activo	Host responde ARP y existe en la LAN
Red local Docker	172.19.0.4	ARP	172.19.0.4 is at ...	Activo	Host responde ARP y existe en la LAN
Red local Docker	172.19.0.5	ARP	Sin respuesta	Inactivo / no existe	No se detecta host
Red local Docker	172.19.0.6	ARP	Sin respuesta	Inactivo / no existe	No se detecta host
Red local Docker	172.19.0.7	ARP	Sin respuesta	Inactivo / no existe	No se detecta host

2. Resultado con ICMP, UDP y TCP:

IP objetivo	Protocolo	Respuesta	Resultado	Interpretación
10.0.1.10	ICMP Timestamp	Timestamp Reply	Activo	Host responde a ICMP
10.0.1.10	UDP puerto 80	ICMP Port Unreachable	Activo	Host existe aunque puerto cerrado
10.0.1.10	TCP ACK puerto 80	TCP RST	Activo	Host activo con pila TCP
10.0.2.10	ICMP/TCP/UDP	Respuesta detectada	Activo	Host remoto accesible

3. Host activos:

Host detectado	Tipo de detección
172.19.0.2	ARP
172.19.0.3	ARP
172.19.0.4	ARP
10.0.1.10	ICMP / UDP / TCP
10.0.2.10	ICMP / UDP / TCP

Parte 2: comportamiento por defecto de nmap:

1. Estados y respuesta TCP:

Estado del puerto	Respuesta observada	Significado
Abierto (Open)	SYN → SYN-ACK	Existe un servicio escuchando en ese puerto
Cerrado (Closed)	SYN → RST-ACK	El host responde, pero no hay servicio activo
Filtrado (Filtered)	Sin respuesta / ICMP unreachable	Firewall o filtro bloquea el acceso

2. Comportamiento por defecto de nmap:

Característica	Resultado
Herramienta utilizada	Nmap 7.99
Escaneo por defecto	TCP SYN Scan (si se ejecuta con privilegios)
Número de puertos escaneados por defecto	1000 puertos más comunes
Resolución DNS	Sí, salvo que se use -n
Descubrimiento de host	Sí, salvo que se use -Pn
Parámetro usado en práctica	-Pn

3. Resultado de nmap localhost:

IP escaneada	Host activo	Puertos escaneados	Resultado
127.0.0.1	Sí	1000	Todos cerrados (antes de iniciar Docker Compose)

4. Servicios expuestos con docker compose:

Contenedor	Servicio	Puerto interno	Puerto publicado en host
lab_ftp	FTP	21	21
lab_ssh	SSH	22	2222
lab_http	HTTP (Nginx)	80	8080

5. Resultado del escaneo final (nmap -Pn -p 21,23,2222,8080 localhost):

Puerto	Estado	Servicio detectado por Nmap	Interpretación
21	Open	ftp	Servicio FTP activo
23	Closed	telnet	No existe servicio escuchando
2222	Open	EtherNetIP-1*	Puerto redirigido al SSH del contenedor
8080	Open	http-proxy*	Puerto redirigido al servidor web

Nmap muestra el nombre asociado por su base de datos estándar, aunque realmente pertenece a servicios de docker que están personalizados.

6. Evidencias en Wireshark:

Puerto	Secuencia observada	Estado deducido
21	SYN → SYN-ACK → RST	Abierto
23	SYN → RST-ACK	Cerrado
2222	SYN → SYN-ACK → RST	Abierto
8080	SYN → SYN-ACK → RST	Abierto

5. Conclusiones:

Gracias a esta práctica me permitió ver y comprender de forma práctica como funciona el descubrimiento de hosts en una red y la diferencia real de trabajar en redes locales y redes externas.

Muchos conceptos no estaban asimilados, solo estaba visto de forma teórica y esto permite visualizarlo de una forma más real.

Dándome una muestra de que no existe solo un método válido para detectar equipos activos, sino que depende del entorno en el que me encuentre, haciendo que en redes locales sea más rápido mediante ARP.

Y en la segunda parte me ayudó a comprobar como funcionaba realmente nmap haciéndome ver lo importante que es la administración de las redes y las auditorías de la seguridad. Es cierto que la herramienta ya era conocida, pero de una forma básica, lo que me ayudó a entrar en profundidad en ella.

6. Bibliografía:

1. Ivanov, I., & Ivanov, I. (n.d.). *TCP and UDP essentials*. NetworkAcademy.IO. <https://www.networkacademy.io/ccna/network-security/tcp-and-udp-essentials>
2. Peiro, E. (2026, April 23). Modelo OSI y TCP/IP: Las 7 Capas Explicadas para Principiantes [2026]. *Aprender21*. <https://www.aprender21.com/blog/modelo-osi-tcp-ip-guia>
3. Docker compose: Inc, D. (2026, January 7). *Docker compose*. Docker Documentation. <https://docs.docker.com/compose/>